

Exercices du cours VSE

Exercices de vérification logicielle

Fuzzing

semestre automne 2025 - 2026

Cet exercice vise à explorer deux frameworks pour le fuzzing *automatique*, cifuzz, clang, AFL++, ainsi que ce qui peut être mis en place en python pour du fuzzing de fichiers XML.

cifuzz

Reprenez le code du dossier cifuzz, et dans un terminal placez-vous dans ce répertoire. Lancez la commande :

```
cifuzz run my_fuzz_test
```

Cette commande compile les sources et lance un test de fuzzing.

Il est également possible d'exploiter le `CMakeLists.txt` présent dans le répertoire :

```
mkdir build
cd build
cmake ..
make
./cifuzz_example          # Lance le programme correspondant à main.cpp
./my_fuzz_test             # Lance le programme de fuzzing, mais sans fuzzing
cifuzz run my_fuzz_test    # Lance les tests avec fuzzing
```

Vous pouvez modifier le fichier `exploreme.cpp` pour ajouter/enlever des bugs.

clang

clang vient avec un fuzzer. Vous disposez d'un exemple dans le répertoire `clangfuzzing`. Le script `compile.sh` permet de compiler le fichier source. Vous pouvez y trouver à l'intérieur `-DTEST1` qui, si vous le modifiez, permet de tester différentes erreurs (regardez le fichier source pour voir de quoi il s'agit). Lancez à chaque fois l'exécutable. Pour `TEST3` vous devriez observer quelque chose de surprenant.

Faites quelques essais de modification du code pour voir ce qui est détecté ou non.

AFL++

Installation

En ligne de commande, simplement utiliser l'installateur standard :

```
sudo apt install afl
```

Exemples

Avant de pouvoir compiler/exécuter, il faut lancer cette commande :

```
export AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1
```

Pour compiler, il suffit exécuter cette commande

```
afl-clang-fast simple_example.c -o simple_example
afl-clang-fast simple_stack.c -o simple_stack
```

```
afl-fuzz -i seed_example -o out_example -- ./simple_example
afl-fuzz -i seed_stack -o out_stack -- ./simple_stack @@
```

```
american fuzzy lop ++4.34c [default] (./simple_example) [explore]
- process timing                               overall results
  run time      : 0 days, 0 hrs, 0 min, 0 sec    cycles done : 3
  last new find : 0 days, 0 hrs, 0 min, 0 sec    corpus count : 4
  last saved crash : 0 days, 0 hrs, 0 min, 0 sec  saved crashes : 1
  last saved hang : none seen yet                 saved hangs : 0
- cycle progress
  now processing : 3.1 (75.0%)
  run time used  : 0 (0.00%)
- stage progress
  now trying : havoc
  stage execs : 180/200 (90.00%)
  total execs : 1751
  exec speed : 1823/sec
- fuzzing strategy yields
  bit flips : 0/0, 0/0, 0/0
  byte flips : 0/0, 0/0, 0/0
  arithmetics : 0/0, 0/0, 0/0
  known ints : 0/0, 0/0, 0/0
  dictionary : 0/0, 0/0, 0/0, 0/0, 0/0
  havoc/splice : 2/1525, 0/0
  py/custom/rq : unused, unused, unused, unused
  ctrl/eff : 75.00%/7, n/a
- strategy: explore                             state: started :-)
- map coverage
  map density : 0.00% / 0.00%
  count coverage : 1.00 bits/tuple
  findings in depth
    favored items : 2 (50.00%)
    new edges on : 2 (50.00%)
    total crashes : 50 (1 saved)
    total tmouts : 0 (0 saved)
- item geometry
  levels : 2
  pending : 0
  pend fav : 0
  own finds : 1
  imported : 0
  stability : 100.00%
```

Note

XML fuzzing

Avant de pouvoir faire fonctionner le script il faut une ou deux étapes d'installation. Ci-dessous la liste des commandes à faire, dans la VM (je vous laisse adapter pour votre machine le cas échéant), depuis le répertoire `xmlfuzzing` (qui ne doit pas être un dossier partagé avec le host) :

```
sudo apt install python3.10-venv
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

```
source venv/bin/activate
```

Nous vous demandons d'ajouter deux ou trois de ces modifications :

1. Faire que les champs contenant des nombres à virgule soient remplacés par la même valeur mais négative.
2. Faire que les champs numériques (entier ou flottant) soient remplacés par 0.
3. Faire que les champs numériques (entier ou flottant) soient remplacés par 1000 fois leur valeur.
4. Faire que les champs *unit* soient modifiés avec une chaîne aléatoire quelconque (qui n'est pas une unité).
5. Faire que les champs *unit*, s'ils sont un poids, deviennent une concentration, et réciproquement :
 - kg → kg/l
 - mg → mg/l
 - mg/l → mg
6. Faire que les dates soient décalées dans le temps de plus de 2 mois
7. Faire que les dates soient décalées dans le temps de moins de 2 mois

Saurez-vous faire planter le système ?